ASSIGNMENT-2

Course: Full Stack Web Development
Course Code: 23CM4121 **Branch:** CSM **Section:** B
Task No.: Assignment 2
Student Name: P. Govindamma
Roll No.: A24126552104

1. TITLE

Build a Student Record Management System using MongoDB

2. AIM

To develop a Student Record Management System using MongoDB that can store, retrieve, update, and manage student information through CRUD operations, filtering with comparison and logical operators, sorting, indexing, index-performance analysis, and aggregation.

3. OBJECTIVE

The objectives of this assignment are:

- To create a MongoDB database (collegeDB) and a collection (students) inside it.
- To insert ten student documents into the collection using insertMany().
- To display all student records using find().pretty() and to fetch one record using findOne().
- To filter students by branch, by marks above a given value ($gt), and by year of study.
- To combine conditions using $gte/$lte ranges, multiple fields (AND) and $or.
- To search for a student using rollNo and to verify several records at once using $in.
- To update marks, email and branch of different students using updateOne() with $set.
- To delete a student record using deleteOne() and confirm the deletion.
- To display students sorted in descending order of marks.
- To create a unique index on rollNo and list indexes using getIndexes().
- To compare COLLSCAN and IXSCAN on a 100,000-document collection using explain("executionStats").
- To run further real-time queries: marks above 80, marks below 50, the highest scorer, a given branch, sorted marks, and average marks per branch using the aggregation pipeline.

4. THEORY

MongoDB is a NoSQL, document-oriented database. Instead of rows and columns, it stores data as documents - JSON-like records (technically BSON) - grouped into collections inside a

database. A collection does not enforce a fixed schema, so documents in the same collection can have different fields if needed.

use("collegeDB") switches the shell's context to the collegeDB database, creating it the first time data is actually written to it. db.createCollection("students") explicitly creates the students collection inside it.

CRUD operations in MongoDB map onto four kinds of methods:

- Create - insertOne() / insertMany() add one or more documents to a collection.
- Read - find() and findOne() retrieve documents, optionally filtered by a query object, sorted with .sort(), and limited with .limit().
- Update - updateOne() / updateMany() modify fields of matching documents, typically using the $set operator to change specific fields without touching the rest of the document.
- Delete - deleteOne() / deleteMany() remove documents that match a filter.

Query operators let a filter compare values instead of only matching exactly: $gt (greater than), $gte (greater than or equal), $lt (less than), $lte (less than or equal), $in (matches any value in a list) and $or (matches if any one of several conditions is true). Listing several fields in one filter, such as { year: 3, marks: { $gt: 80 } }, acts as a logical AND.

Aggregation: the aggregate() method passes documents through a pipeline of stages. $group collects documents by a key (here, branch) and computes values such as $avg and $sum for each group, and $sort orders the grouped results.

Indexing: without an index, MongoDB must perform a collection scan (COLLSCAN) - reading every single document to check whether it matches a query - which gets slower as the collection grows. An index, created with createIndex(), builds a separate, sorted data structure (a B-tree) on the indexed field. A query that filters or sorts on an indexed field can then use an index scan (IXSCAN) to jump almost directly to the matching documents. Adding { unique: true } also stops two documents from sharing the same rollNo. explain("executionStats") reports the plan used and how many documents were examined, which is used here to compare the two cases on a 100,000-document collection.

5. HOW TO BUILD THIS PROJECT - STEP-BY-STEP SETUP GUIDE

Install MongoDB Community Server

Download and install MongoDB Community Server for your OS from mongodb.com. On Windows/macOS the installer also offers to install MongoDB Compass (a GUI) alongside it.

Install the MongoDB Shell (mongosh)

mongosh is the modern command-line shell used to run JavaScript-style queries against MongoDB. It is bundled with recent installers, or can be installed separately from the MongoDB Download Center.

Start the MongoDB server (mongod)

Run the mongod process so a database server is listening (by default on port 27017). On Windows this usually runs as a service automatically; on macOS/Linux it may need to be started manually, e.g. mongod --dbpath /path/to/data.

Connect using mongosh

Open a terminal and run mongosh. This connects to the local MongoDB server and opens an interactive JavaScript-style shell.

Switch to (create) the database

Run use("collegeDB"). MongoDB does not create the database on disk until the first document is inserted.

Create the students collection

Run db.createCollection("students"). Then run db.students.deleteMany({}) so the script can be re-run safely from an empty collection.

Insert sample student documents

Run db.students.insertMany([...]) with an array of ten student objects.

Run read, update, and delete queries

Use find(), findOne(), updateOne(), and deleteOne() with the filters and operators described in this report.

Create an index and verify it

Run db.students.createIndex({ rollNo: 1 }, { unique: true }) and then db.students.getIndexes() to confirm the index exists alongside the default _id index.

Measure the effect of the index

Fill a studentsBig collection with 100,000 documents using a JavaScript loop, run explain("executionStats") before and after creating the index, and compare the stage and documents examined.

Save the script and re-run it

Save all the commands in mongo_script.js and run the whole file with: mongosh collegeDB mongo_script.js

(Optional) Inspect the data visually

Open MongoDB Compass and connect to localhost:27017 to browse the collegeDB database and the students collection.

6. ALGORITHM / PROCEDURE

- Switch to the collegeDB database with use("collegeDB"), create the students collection and clear any old data with deleteMany({}).
- Insert ten student documents in one call using insertMany(), each with rollNo, name, branch, year, marks, and email.

- Display every document using find().pretty(), then filter by branch "CSE-AIML" and by marks greater than 75 ($gt).
- Search for rollNo "23CM004" using find() and findOne(), and filter by year equal to 3.
- Run combined filters: marks between 60 and 80 ($gte, $lte), year 3 with marks above 80 (AND), and branch IT or marks below 50 ($or).
- Update marks of 23CM005 to 55, the email of 23CM003, and the branch of 23CM007 using updateOne() with $set; verify all three with $in.
- Delete the document with rollNo "23CM009" using deleteOne() and confirm with find().
- Retrieve all documents sorted by marks in descending order using .sort({ marks: -1 }).
- Create a unique ascending index on rollNo with createIndex(), then list indexes with getIndexes().
- Create studentsBig with 100,000 generated documents; run explain("executionStats") before the index (COLLSCAN), create the index, and run it again (IXSCAN).
- Run the real-time queries: marks above 80, marks below 50, highest scorer (sort + limit(1)), branch CSE, all students sorted by marks, and average marks per branch using aggregate().

7. PROGRAM

mongo_script.js (run with mongosh)

```
use("collegeDB");


db.createCollection("students");


db.students.deleteMany({});


db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar",    branch: "CSE-AIML", year: 3, marks: 85, email:
"ravi@example.com" },   { rollNo: "23CM002", name: "Sneha Reddy",   branch: "CSE-AIML",
year: 3, marks: 92, email:
"sneha@example.com" },   { rollNo: "23CM003", name: "Arjun Varma",   branch: "CSE",
year: 2, marks: 68, email:
"arjun@example.com" },   { rollNo: "23CM004", name: "Priya Sharma",  branch: "ECE",
year: 3, marks: 77, email:
"priya@example.com" },   { rollNo: "23CM005", name: "Kiran Babu",    branch: "CSE",
year: 4, marks: 45, email:
"kiran@example.com" },   { rollNo: "23CM006", name: "Lakshmi Devi",  branch: "IT",
year: 2, marks: 88, email:
"lakshmi@example.com" },   { rollNo: "23CM007", name: "Manoj Naidu",   branch: "ECE",
year: 1, marks: 59, email:
"manoj@example.com" },   { rollNo: "23CM008", name: "Divya Rao",     branch: "CSE-AIML",
year: 2, marks: 73, email:
"divya@example.com" },   { rollNo: "23CM009", name: "Suresh Raju",   branch: "IT",
year: 4, marks: 48, email:
"suresh@example.com" },   { rollNo: "23CM010", name: "Anitha Kumari", branch: "CSE",
year: 3, marks: 81, email:
"anitha@example.com" }
]);


db.students.find().pretty();


db.students.find({ branch: "CSE-AIML" });
```

```
db.students.find({ marks: { $gt: 75 } });

db.students.find({ rollNo: "23CM004" });

db.students.findOne({ rollNo: "23CM004" });

db.students.find({ year: 3 });
```

```
db.students.find({ marks: { $gte: 60, $lte: 80 } });
db.students.find({ year: 3, marks: { $gt: 80 } });
db.students.find({ $or: [{ branch: "IT" }, { marks: { $lt: 50 } }] });


db.students.updateOne({ rollNo: "23CM005" }, { $set: { marks: 55 } });


db.students.updateOne({ rollNo: "23CM003" }, { $set: { email: "arjun.varma@example.com" } });
db.students.updateOne({ rollNo: "23CM007" }, { $set: { branch: "CSE" } });
db.students.find({ rollNo: { $in: ["23CM003", "23CM005", "23CM007"] } });


db.students.deleteOne({ rollNo: "23CM009" });
db.students.find({ rollNo: "23CM009" });


db.students.find().sort({ marks: -1 });


db.students.createIndex({ rollNo: 1 }, { unique: true });
db.students.getIndexes();


db.studentsBig.drop();
const bulk = [];
for (let i = 1; i <= 100000; i++) {
  bulk.push({
    rollNo: "R" + String(i).padStart(6, "0"),
    name: "Student " + i,
    branch: ["CSE", "CSE-AIML", "ECE", "IT"][i % 4],
    year: (i % 4) + 1,
    marks: Math.floor(Math.random() * 101),
email: "student" + i + "@example.com"
  });
}
db.studentsBig.insertMany(bulk);


// (a) WITHOUT index -> COLLSCAN: scans all 100000 documents
let before = db.studentsBig.find({ rollNo: "R099999" }).explain("executionStats");
print("WITHOUT INDEX");
print("  stage              :", before.queryPlanner.winningPlan.stage);
print("  docs examined      :", before.executionStats.totalDocsExamined);
print("  time (ms)          :", before.executionStats.executionTimeMillis);


// (b) Create the index
db.studentsBig.createIndex({ rollNo: 1 });


// (c) WITH index -> IXSCAN: examines just 1 document
let after = db.studentsBig.find({ rollNo: "R099999" }).explain("executionStats");
print("WITH INDEX");
print("  stage              :", after.queryPlanner.winningPlan.stage,
      "->", after.queryPlanner.winningPlan.inputStage.stage);
print("  docs examined      :", after.executionStats.totalDocsExamined);
print("  time (ms)          :", after.executionStats.executionTimeMillis);


db.students.find({ marks: { $gt: 80 } });


db.students.find({ marks: { $lt: 50 } });
db.students.find().sort({ marks: -1 }).limit(1);


db.students.find({ branch: "CSE" });


db.students.find().sort({ marks: -1 });


db.students.aggregate([
```

```
  { $group: { _id: "$branch", avgMarks: { $avg: "$marks" }, count: { $sum: 1 } } },
  { $sort: { avgMarks: -1 } }
]);
```

8. CODE EXPLANATION

Code	Explanation

use("collegeDB")	Switches the shell to the collegeDB database, creating it on first write.
db.createCollection("students")	Explicitly creates the students collection.
db.students.deleteMany({})	Removes all existing documents so the script starts clean every time.
db.students.insertMany([...])	Inserts an array of ten documents in one call.
find().pretty()	Returns every document, formatted for easy reading.
find({ branch: "CSE-AIML" })	Exact match on the branch field.
find({ marks: { $gt: 75 } })	Marks strictly greater than 75.
find({ rollNo: ... }) / findOne()	find() returns a cursor of matches; findOne() returns the first matching document only.
find({ year: 3 })	Uses a different field (year) as the filter.
{ $gte: 60, $lte: 80 }	Range query: marks from 60 up to and including 80.
find({ year: 3, marks: { $gt: 80 } })	Two conditions in one filter - both must be true (AND).
$or: [...]	Matches documents satisfying at least one of the listed conditions.
updateOne(filter, { $set: {...} })	Finds the first document matching filter and sets the given field(s).
rollNo: { $in: [...] }	Matches any document whose rollNo is in the list.
deleteOne({ rollNo: "23CM009" })	Deletes the first document matching the filter.
find().sort({ marks: -1 })	All documents ordered by marks, highest first (-1 = descending).
createIndex({ rollNo: 1 }, { unique: true })	Builds an ascending index on rollNo and rejects duplicate roll numbers.
getIndexes()	Lists every index on the collection.
studentsBig loop + insertMany(bulk)	Generates 100,000 documents in memory and inserts them in one call.
explain("executionStats")	Reports the winning plan stage, documents examined and execution time.
.limit(1)	Restricts the sorted result to the first (highest-scoring) document.
aggregate([$group, $sort])	Groups students by branch, computes average marks and count, then sorts by average (highest first).

9. TEST CASES AND EXECUTION DATA

Each command was executed in order against the collegeDB database and the result was recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	db.createCollection("students")	Collection is created	{ ok: 1 }	Pass

Test Case	Action	Expected Result	Actual Result	Status
TC-02	insertMany() with 10 documents	All 10 inserted	acknowledged: true, 10 insertedIds	Pass
TC-03	find().pretty()	All 10 documents returned	10 documents returned	Pass
TC-04	find({ branch: "CSE-AIML" })	Ravi, Sneha, Divya	3 documents returned	Pass
TC-05	find({ marks: { $gt: 75 } })	Students above 75	5 documents (85, 92, 77, 88, 81)	Pass
TC-06	find({ rollNo: "23CM004" }) / findOne()	Priya Sharma's document	1 document returned	Pass
TC-07	find({ year: 3 })	Year 3 students	4 documents returned	Pass
TC-08	marks $gte 60 and $lte 80	Marks in 60-80	3 documents (68, 77, 73)	Pass
TC-09	year 3 AND marks $gt 80	Ravi, Sneha, Anitha	3 documents returned	Pass
TC-10	$or: branch IT / marks < 50	Kiran, Lakshmi, Suresh	3 documents returned	Pass
TC-11	updateOne() - marks of 23CM005 to 55	Marks updated	matchedCount: 1, modifiedCount: 1	Pass
TC-12	updateOne() - email of 23CM003	Email updated	matchedCount: 1, modifiedCount: 1	Pass
TC-13	updateOne() - branch of 23CM007 to CSE	Branch updated	matchedCount: 1, modifiedCount: 1	Pass
TC-14	find({ rollNo: { $in: [...] } })	3 updated records shown	3 documents with new values	Pass
TC-15	deleteOne({ rollNo: "23CM009" })	Record deleted	deletedCount: 1	Pass
TC-16	find({ rollNo: "23CM009" })	No record found	[] (empty)	Pass
TC-17	find().sort({ marks: -1 })	Highest marks first	Sneha (92) first, Kiran (55) last	Pass
TC-18	createIndex({ rollNo: 1 }, { unique: true })	Unique index created	rollNo_1	Pass
TC-19	getIndexes()	_id and rollNo indexes	2 indexes: _id_ and rollNo_1	Pass

TC-20	explain() without index (100,000 docs)	COLLSCAN	COLLSCAN, 100000 docs examined	Pass
TC-21	explain() with index	IXSCAN	FETCH -> IXSCAN, 1 doc examined	Pass
TC-22	find({ marks: { $gt: 80 } })	Students above 80	4 documents (85, 92, 88, 81)	Pass
TC-23	find({ marks: { $lt: 50 } })	Students below 50	[] - none remain	Pass
TC-24	sort + limit(1)	Highest scorer	Sneha Reddy (92)	Pass
TC-25	find({ branch: "CSE" })	CSE students	4 documents returned	Pass
TC-26	aggregate() average per branch	Four branch groups, sorted	IT 88, CSE-AIML 83.33, ECE 77, CSE 65.75	Pass

10. OUTPUT

Output 1: Create the students collection

```
Terminal - mongosh


collegeDB> db.createCollection("students") { ok: 1 }
```

Output 2: Insert 10 student documents

```
Terminal - mongosh


collegeDB> db.students.insertMany([ ... ])
{
  acknowledged: true,
insertedIds: {
    '0': ObjectId('657f1a2b3c4d5e6f7a8b9c01'),
    '1': ObjectId('657f1a2b3c4d5e6f7a8b9c02'),
    '2': ObjectId('657f1a2b3c4d5e6f7a8b9c03'),
    '3': ObjectId('657f1a2b3c4d5e6f7a8b9c04'),
    '4': ObjectId('657f1a2b3c4d5e6f7a8b9c05'),
    '5': ObjectId('657f1a2b3c4d5e6f7a8b9c06'),
    '6': ObjectId('657f1a2b3c4d5e6f7a8b9c07'),
    '7': ObjectId('657f1a2b3c4d5e6f7a8b9c08'),
    '8': ObjectId('657f1a2b3c4d5e6f7a8b9c09'),
    '9': ObjectId('657f1a2b3c4d5e6f7a8b9c10')
  }
}
```

Output 3: Display all students

```
Terminal - mongosh


collegeDB> db.students.find().pretty()
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },   { rollNo:
```

```
'23CM005', name: 'Kiran Babu', branch: 'CSE',     year: 4,
marks: 45, email: 'kiran@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch: 'IT',
year: 2, marks: 88, email: 'lakshmi@example.com' },   { rollNo:
'23CM007', name: 'Manoj Naidu', branch: 'ECE',     year: 1,
marks: 59, email: 'manoj@example.com' },
  { rollNo: '23CM008', name: 'Divya Rao', branch: 'CSE-AIML',
year: 2, marks: 73, email: 'divya@example.com' },   { rollNo:
'23CM009', name: 'Suresh Raju', branch: 'IT',     year: 4,
marks: 48, email: 'suresh@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' } ]
```

Output 4: Students in branch CSE-AIML

```
Terminal - mongosh
```

```
collegeDB> db.students.find({ branch: "CSE-AIML" })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM008', name: 'Divya Rao', branch: 'CSE-AIML',
year: 2, marks: 73, email: 'divya@example.com' } ]
```

Output 5: Students scoring more than 75

```
Terminal - mongosh


collegeDB> db.students.find({ marks: { $gt: 75 } })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch: 'IT',
year: 2, marks: 88, email: 'lakshmi@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' } ]
```

Output 6: Search student by rollNo

```
Terminal - mongosh


collegeDB> db.students.find({ rollNo: "23CM004" })
[
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' } ]
```

Output 7: Search a single student with findOne()

```
Terminal - mongosh
```

```
collegeDB> db.students.findOne({ rollNo: "23CM004" }) { rollNo:
'23CM004', name: 'Priya Sharma', branch: 'ECE',   year: 3,
marks: 77, email: 'priya@example.com' }
```

Output 8: Students in year 3 (condition-based search)

```
Terminal - mongosh


collegeDB> db.students.find({ year: 3 })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' }
]
```

Output 9: Marks between 60 and 80 ($gte / $lte)

```
Terminal - mongosh


collegeDB> db.students.find({ marks: { $gte: 60, $lte: 80 } })
[
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },
  { rollNo: '23CM008', name: 'Divya Rao', branch: 'CSE-AIML',
year: 2, marks: 73, email: 'divya@example.com' } ]
```

Output 10: Year 3 students with marks above 80 (AND condition)

```
Terminal - mongosh


collegeDB> db.students.find({ year: 3, marks: { $gt: 80 } })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' } ]
```

Output 11: Branch IT or marks below 50 ($or)

```
Terminal - mongosh


collegeDB> db.students.find({ $or: [{ branch: "IT" }, { marks: { $lt: 50 } }] })
[
  { rollNo: '23CM005', name: 'Kiran Babu', branch: 'CSE',
year: 4, marks: 45, email: 'kiran@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch:
'IT',     year: 2, marks: 88, email:
'lakshmi@example.com' },   { rollNo: '23CM009', name:
'Suresh Raju', branch: 'IT',     year: 4, marks: 48,
email: 'suresh@example.com' }
]
```

Output 12: Update marks for rollNo 23CM005

```
Terminal - mongosh


collegeDB> db.students.updateOne({ rollNo: "23CM005" }, { $set: { marks: 55 } })
{
  acknowledged: true,
insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0
}
```

Output 13: Update email for rollNo 23CM003

```
Terminal - mongosh
 collegeDB> db.students.updateOne({ rollNo: "23CM003" }, { $set: {
email:
"arjun.varma@example.com" } })
{
  acknowledged: true,
insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0
}
```

Output 14: Update branch for rollNo 23CM007

```
Terminal - mongosh


collegeDB> db.students.updateOne({ rollNo: "23CM007" }, { $set: { branch: "CSE" } })
{
  acknowledged: true,
insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0
}
```

Output 15: Verify updates using $in

```
Terminal - mongosh


collegeDB> db.students.find({ rollNo: { $in: ["23CM003", "23CM005", "23CM007"] } })
[
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun.varma@example.com' },   {
rollNo: '23CM005', name: 'Kiran Babu', branch: 'CSE',
year: 4, marks: 55, email: 'kiran@example.com' },
  { rollNo: '23CM007', name: 'Manoj Naidu', branch: 'CSE',
year: 1, marks: 59, email: 'manoj@example.com' }
]
```

Output 16: Delete student by rollNo (23CM009)

```
Terminal - mongosh


collegeDB> db.students.deleteOne({ rollNo: "23CM009" }) {
acknowledged: true, deletedCount: 1 }
```

Output 17: Confirm deletion

```
Terminal - mongosh


collegeDB> db.students.find({ rollNo: "23CM009" }) []
```

Output 18: Students sorted by marks (descending)

```
Terminal - mongosh
```

```
collegeDB> db.students.find().sort({ marks: -1 })
[
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
    year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch: 'IT',
year: 2, marks: 88, email: 'lakshmi@example.com' },
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
    year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },
  { rollNo: '23CM008', name: 'Divya Rao', branch: 'CSE-AIML',
    year: 2, marks: 73, email: 'divya@example.com' },
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun.varma@example.com' },   {
rollNo: '23CM007', name: 'Manoj Naidu', branch: 'CSE',
year: 1, marks: 59, email: 'manoj@example.com' },
  { rollNo: '23CM005', name: 'Kiran Babu', branch: 'CSE',
year: 4, marks: 55, email: 'kiran@example.com' }
]
```

Output 19: Create unique index on rollNo

```
Terminal - mongosh


collegeDB> db.students.createIndex({ rollNo: 1 }, { unique: true })
rollNo_1
```

Output 20: List all indexes on the collection

```
Terminal - mongosh


collegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1', unique: true }
]
```

Output 21: Index performance on 100,000 documents (studentsBig)

```
Terminal - mongosh


collegeDB> load('mongo_script.js')  // performance section
WITHOUT INDEX
  stage              : COLLSCAN
docs examined      : 100000
  time (ms)          : [fill from your run]
WITH INDEX
  stage              : FETCH -> IXSCAN
  docs examined      : 1
  time (ms)          : [fill from your run]
```

Output 22: (Real-Time) Students scoring above 80

```
Terminal - mongosh


collegeDB> db.students.find({ marks: { $gt: 80 } })
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
    year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch: 'IT',
year: 2, marks: 88, email: 'lakshmi@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' } ]
```

Output 23: (Real-Time) Students scoring below 50

```
Terminal - mongosh


collegeDB> db.students.find({ marks: { $lt: 50 } })
[]
// no student is below 50 now: 23CM005 was raised to 55 and
23CM009 was deleted
```

Output 24: (Real-Time) Highest-scoring student

```
Terminal - mongosh


collegeDB> db.students.find().sort({ marks: -1 }).limit(1)
[
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' } ]
```

Output 25: (Real-Time) Students from branch CSE

```
Terminal - mongosh


collegeDB> db.students.find({ branch: "CSE" })
[
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun.varma@example.com' },   {
rollNo: '23CM005', name: 'Kiran Babu', branch: 'CSE',     year:
4, marks: 55, email: 'kiran@example.com' },
  { rollNo: '23CM007', name: 'Manoj Naidu', branch: 'CSE',
year: 1, marks: 59, email: 'manoj@example.com' },
```

```
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' } ]
```

Output 26: (Real-Time) Students sorted by marks

```
Terminal - mongosh


collegeDB> db.students.find().sort({ marks: -1 })
[
  { rollNo: '23CM002', name: 'Sneha Reddy', branch: 'CSE-AIML',
year: 3, marks: 92, email: 'sneha@example.com' },
  { rollNo: '23CM006', name: 'Lakshmi Devi', branch: 'IT',
year: 2, marks: 88, email: 'lakshmi@example.com' },
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML',
year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM010', name: 'Anitha Kumari', branch: 'CSE',
year: 3, marks: 81, email: 'anitha@example.com' },
  { rollNo: '23CM004', name: 'Priya Sharma', branch: 'ECE',
year: 3, marks: 77, email: 'priya@example.com' },
  { rollNo: '23CM008', name: 'Divya Rao', branch: 'CSE-AIML',
    year: 2, marks: 73, email: 'divya@example.com' },
  { rollNo: '23CM003', name: 'Arjun Varma', branch: 'CSE',
year: 2, marks: 68, email: 'arjun.varma@example.com' },   {
rollNo: '23CM007', name: 'Manoj Naidu', branch: 'CSE',
year: 1, marks: 59, email: 'manoj@example.com' },
  { rollNo: '23CM005', name: 'Kiran Babu', branch: 'CSE',
year: 4, marks: 55, email: 'kiran@example.com' } ]
```

Output 27: (Real-Time) Average marks per branch (aggregation)

```
Terminal - mongosh


collegeDB> db.students.aggregate([ { $group: ... }, { $sort: {
avgMarks: -1 } } ])
[
  { _id: 'IT', avgMarks: 88, count: 1 },
  { _id: 'CSE-AIML', avgMarks: 83.33333333333333, count: 3 },
  { _id: 'ECE', avgMarks: 77, count: 1 },
  { _id: 'CSE', avgMarks: 65.75, count: 4 }
]
```

11. RESULT

Thus, a Student Record Management System was successfully built using MongoDB. The students collection was created, populated with ten records, queried by branch, marks, rollNo and year using comparison and logical operators, updated, sorted, indexed with a unique index, and a student record was deleted. The index comparison on 100,000 documents showed a COLLSCAN examining every document versus an IXSCAN examining one, and the real-time queries and aggregation were completed.